



SASKEN

To Automate or Not to Automate

Abstract:

With the advancement of technology, significant investment in research and development, and increased production capacity, we are constantly surrounded by more sophisticated and novel objects. Consumers' expectations are evolving. We desire instant access to better and more recent items. Additionally, this has compelled the market, whether it be for hardware or software, to be extremely competitive in terms of cost, value, and time to market. Only with a well-defined process and infrastructure in place that can deliver more quickly, accurately, and affordably can this be accomplished. So, the solution is automation.

Author:

Debabrata Sahoo, Principal Engineer - App Software Test
Sasken Technologies Ltd.



Table of Content

Introduction	4
Clear understanding of the device or SW under test	5
Some of the deciding factors for automation	6
Some of the deciding factors for the automation tool are	7
Conclusion	9
About the Author	9
About Sasken	9



Introduction

To ensure software quality, automation is essential. It is helpful in a variety of ways, among them:

- Higher quality
- Test accuracy
- Ability to run tests that would otherwise be impossible manually
- Shorter cycle times
- Lower costs

However, automation must be carefully planned. Like every project, it must go through all phases of planning, including analyzing requirements, determining scope, conducting a feasibility assessment, and carrying out the plan. In the absence of them, some of the previously listed advantages change into drawbacks, resulting in failure and derailing the completion of the entire project.

Some of the prevalent myths:

- Every project must have automation.
- Automation gives substantial cost savings.
- Complete manual testing can be replaced by automation.
- Automation is a one time expense.

You may see a conflict between the viewpoints expressed above up to this point. However, the paper tries to thoroughly clarify various viewpoints and connect the ideas.

Beginning with the automation process:

- What to automate; the major goal of this is to comprehend the device being tested and to examine all the test cases.
- Identify a tool that is best suited for this need.
- Identify the test cases that can be automated.
- Plan when we need to start automating
- Plan when we need to execute the automated test.



Clear understanding of the device or SW under test

This is important in determining the automation strategy. In the end, automation involves communicating with the system through an interface that is available by the device or software under test. Because there may be one or more interfaces, it is critical to understand what is possible and what is not. Some things to keep an eye out for are:

- Are the interfaces well defined?
- Are the interfaces stable enough?
- Are the interfaces going to be fixed or under constant modification?
- Is the interface standardized?
- Does the system allow the installation of new SW (test agents) or modification of actual systems under test to provide additional support as needed?

It is important to know whether the system is designed to support automation.

Automation planning must be done from the start of the SW or product's Design and Development phase, not at the validation stage.

Planning automation beyond what the built-in interfaces can allow can only lead to long-term failures and a negative return on investment (ROI) due to ongoing maintenance.

Examples of interfaces include:

- ADB and UI Automator for Android
- Ethernet for IVI systems
- Shell commands for UNIX-based devices.
- Ether CAT for industrial devices
- Robotic arm

What falls under the testing and automation scope?

We need a clear understanding of the scope to comprehend this. Does the system as a whole or only a portion of it need to be automated?



Some of the deciding factors for automation

- What percentage of test cases can be automated.
- What is the effort required to automate the test cases?
- What person month of manual test effort will be saved.
- Is it difficult to perform the test manually?
 - » Need multiple repetitions such as - stability
 - » Time-sensitive steps: strict response time
- Prone to human error
 - » Complex steps
 - » Complex/multiple validation
- Tests need to be executed in different time zones or need continuous execution during non-working hours.
- GUI interface is not available.
- Complex test setup involving multiple entities (device, cloud, simulator, etc.)



Some of the deciding factors for the automation tool are

Once the test cases that need to be automated have been discovered, choosing the right tool is essential. All the test steps should be supported by the tool's interfaces, which should also allow for validation.

Most tools and automation that rely on API calls, command-based control, response-based validation, or log analysis-based validation are more reliable since they are selected during system design and either stay stable or undergo minimal change.

Even at the conclusion of the project's lifespan, the UI tends to change quite regularly. Simple modifications to the screen's aspect ratio, orientation, form factor, color, language, display density, etc. will result in several failures and require extensive revision of the test script, limiting the benefits of automation. It is also challenging to pinpoint UI-related problems.

Let us go back to the myths pointed out earlier:

- **Every project must have automation.**

Some of us think automation can be done for any project and ought to be done. We need to be aware of some of the fundamental prerequisites for automation, such as the requirement that the system being tested be stable. As automation entails repeating the procedures in a prescribed manner to obtain the same results repeatedly, if the system is unstable, the features will be subject to continuous development, making it impossible to correct the test steps. The automation scripts are destined

to crash every time they are run on fresh builds if changes keep occurring with every build. This may occur because of feature modifications or a steady addition of new features in an agile environment. This will lead to excessive analysis, debugging, and script modification, all of which will require additional work. This occurs when tests require steps and validation that are not API-based. Therefore, automation should be implemented where requirements and designs are fixed, and the system has been manually tested and is functioning properly. The alterations must be kept to a minimum and thoroughly recorded. Otherwise, if accepted blindly without taking them into account, the automation will be overworked.

- **Automation gives substantial cost savings:**

- » It is important to keep in mind that, even though automation reduces test execution time and increases coverage, none of these effects are caused by its speed. There are numerous factors that contribute to the shortening of time. Automation happens constantly, however, an engineer can only undertake manual execution for a brief time, let us say 8 to 9 hours. So the unutilised time (ex- non working hours) can now also be utilised for automation resulting in a reduction in total time.
- » Due to the frequent release of new software and the short amount of time available for testing, all test cases cannot be manually completed for every build. By using automation testing, all test cases may be run simultaneously, increasing build coverage.



- » Scale-up is made possible by automation; it may not be possible to ramp up or down the number of engineers in an abbreviated period. Automation setups, however, may be simply scaled up and down.
- » If we don't have a scope of utilizing the system effectively, due to what soever reason like- less no of devices, lack of builds or setup issue or lack of Enggs with desired capabilities, then adding automation just creates one more over-head instead of giving any benefit.

In practice, automation requires a significant amount of scripting, continual maintenance, regular test result analysis, etc. For this, a distinct group of competent engineers is needed, as well as considerable work (and a higher price) based on automation volume. Which brings up the question of whether automation is expensive. Maybe not. Costly for both initial planning and implementation. However, the average cost per execution eventually goes down if the automation is used more than once. However, this needs to be assessed once more based on how many times we can ideally reuse and how much upkeep will cost.

- **Complete manual testing can be replaced by automation**

We must realize that testing is a way of thinking; automation merely entails repeating a set of predetermined processes while evaluating previous iterations. There are always test cases which require human intervention or require more effort to automate. Expert testing/Random testing is something which is performed by experienced testers, these have to be done manually. There are situations where the system is undergoing continuous change, in these cases it is always advised to defer automation to avoid rework.

- **Automation is a one time expense.**

Regular maintenance of test scripts is necessary, to take care of OS, version, product changes, etc. This effort needs to be factored while doing ROI calculations.

At the end, we must consciously decide whether to automate based on all the factors we have just mentioned. There is not a single recipe that can meet every need. It is important to consider whether automation can identify as many problems as manual execution. Is automation necessary?



Conclusion

A simple rule of thumb for automation is to automate if there is a favorable return on investment.

However, “ROI” should not merely involve effort or outright expense. Customer happiness, quality, accuracy, and time can all contribute to ROI. Which results in income, profit, business expansion, and many other positive outcomes. It has a wide range of facets.

In conclusion, we should grasp automation clearly and know what we want to achieve from it.

About the Author

Debabrata Sahoo, the principal engineer in charge of testing, has been working in the IT sector for 15 years. has worked in a variety of industries, including devices, automotive, IOT, and NW. possess broad experience with a variety of platforms and technologies, including Android smartphones, automotive IVI, Network Simulators, and IOT gateways.

About Sasken

Sasken is a specialist in Product Engineering and Digital Transformation providing concept-to market, chip-to-cognition R&D services to global leaders in Semiconductor, Automotive, Industrials, Consumer Electronics, Enterprise Devices, SatCom, Telecom, and Transportation industries. For over 32 years and with multiple patents, Sasken has transformed the businesses of 100+ Fortune 500 companies, powering more than a billion devices through its services and IP.





To Automate or Not to Automate

marketing@sasken.com | www.sasken.com

USA | FINLAND | GERMANY | JAPAN | INDIA

© **Sasken Technologies Ltd.** All rights reserved.

Products and services mentioned herein are trademarks and service marks of **Sasken Technologies Ltd.**, or the respective companies.